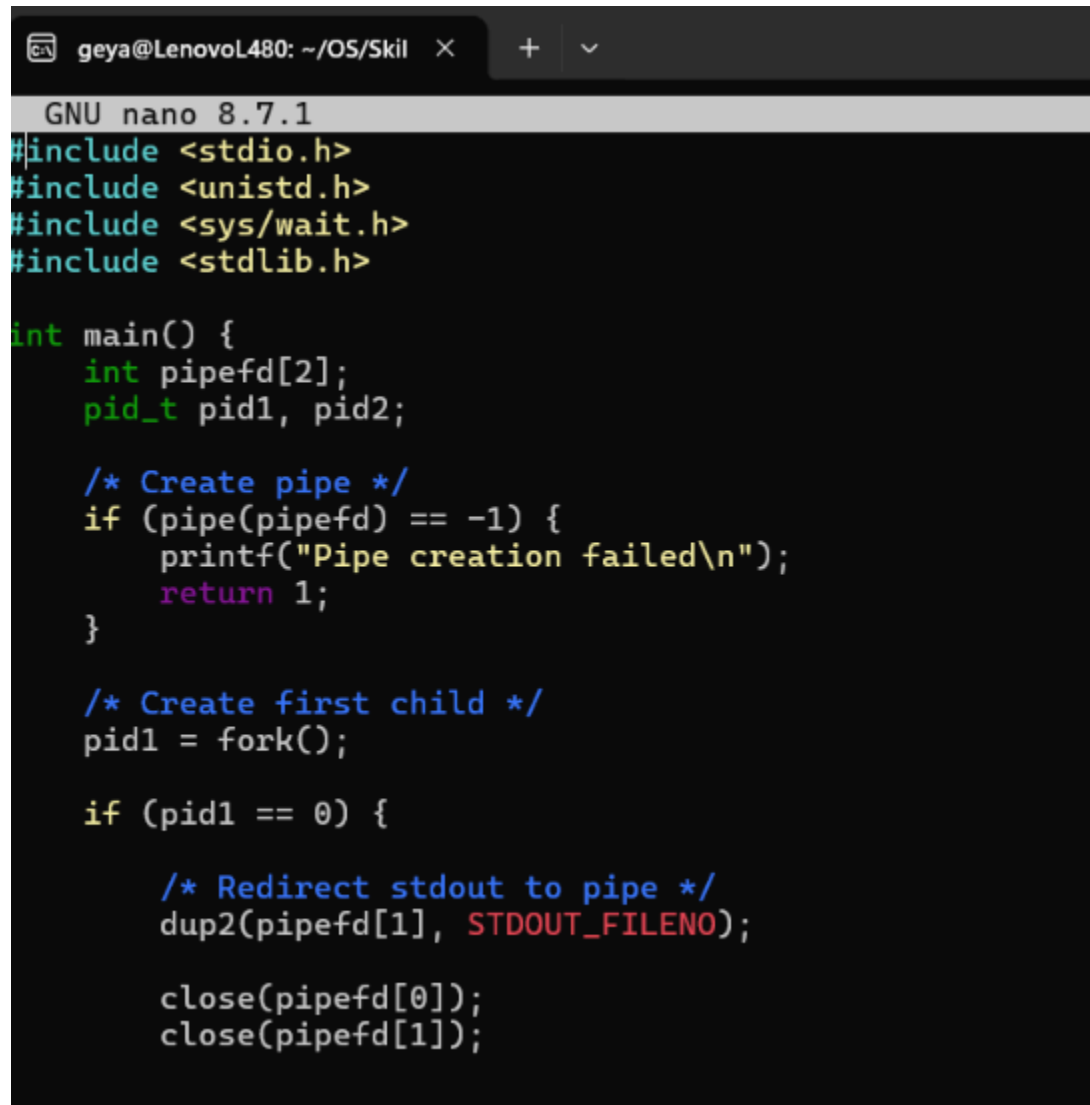


To Create Pipes, Redirect Streams, Execute Connected Commands, Close Descriptors, Synchronize Processes, Test Data Flow

Code:

A screenshot of a terminal window with a dark background. The window title bar shows 'geya@LenovoL480: ~/OS/Skil' and standard window controls. The terminal displays the GNU nano 8.7.1 editor. The code is a C program that creates a pipe, forks a child process, and redirects the child's stdout to the pipe. The code is as follows:

```
GNU nano 8.7.1
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>

int main() {
    int pipefd[2];
    pid_t pid1, pid2;

    /* Create pipe */
    if (pipe(pipefd) == -1) {
        printf("Pipe creation failed\n");
        return 1;
    }

    /* Create first child */
    pid1 = fork();

    if (pid1 == 0) {
        /* Redirect stdout to pipe */
        dup2(pipefd[1], STDOUT_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);
    }
}
```

```

        execlp("ls", "ls", NULL);

        printf("Error executing ls\n");
        exit(1);
    }

    /* Create second child */
    pid2 = fork();

    if (pid2 == 0) {

        /* Redirect stdin from pipe */
        dup2(pipefd[0], STDIN_FILENO);

        close(pipefd[0]);
        close(pipefd[1]);

        execlp("wc", "wc", "-l", NULL);

        printf("Error executing wc\n");
        exit(1);
    }

    /* Parent closes both ends */
    close(pipefd[0]);

```

```

    /* Parent closes both ends */
    close(pipefd[0]);
    close(pipefd[1]);

    /* Wait for children */
    waitpid(pid1, NULL, 0);
    waitpid(pid2, NULL, 0);

    printf("Pipeline completed.\n");

    return 0;
}

```

Output:

```
geya@LenovoL480:~/OS/Skill/Week 6$ ./task3
6
Pipeline completed.
geya@LenovoL480:~/OS/Skill/Week 6$ |
```